Non-Parametric Density Estimation, Metric Spaces, and Asymptotic Bounds

A Unified Mathematical, Information-Theoretic, and Physical Foundation of k-Nearest Neighbors (k-NN)

Contents

1 The Non-Parametric Learning Paradigm and Instance-Based Memory

In parametric machine learning frameworks (such as Logistic Regression, Linear Discriminant Analysis, or Neural Networks), training compresses a dataset $\mathcal{D}$ into a fixed vector of parameters $\boldsymbol{\theta} \in \mathbb{R}^p$. Once $\boldsymbol{\theta}$ is optimized, the raw training data is discarded. The capacity of a parametric model is strictly bounded by the dimensionality p of its parameter space.

In contrast, **k-Nearest Neighbors (k-NN)** belongs to the family of **Non-Parametric, Instance-Based (Lazy) Learning** algorithms.

Definition 1.1 (**Non-Parametric Model**)**.** A learning algorithm is non-parametric if the effective number of parameters grows dynamically with the size of the training dataset N. Rather than assuming a functional form for $P(Y \mid \mathbf{X})$, a non-parametric model uses the data samples themselves to construct a local approximation of the conditional probability density.

1.1 Memory-Based Inference vs. Structural Training

The operational workflow of k-NN fundamentally differs from parametric algorithms:

- **Training Phase ($O(1)$ Computational Complexity):** k-NN performs no explicit parameter optimization during training. The "fitting" step consists entirely of storing the dataset $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$.
- **Inference Phase ($O(Nd)$ Computational Complexity):** Generalization is deferred until a query point $\mathbf{x} \in \mathbb{R}^d$ is received. The algorithm computes metric distances between $\mathbf{x}$ and all N stored training instances to locate its local neighborhood.

1.2 Local Conditional Probability Estimation

Let $\mathcal{N}_k(\mathbf{x}) \subset \mathcal{D}$ represent the subset of k training points closest to query vector $\mathbf{x}$ under metric distance function $d(\cdot, \cdot)$.

For **Classification**, k-NN models the conditional posterior class probability $P(Y = c \mid \mathbf{X} = \mathbf{x})$ as the fraction of local neighbors belonging to class c:

$$P(Y = c \mid \mathbf{X} = \mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbb{I}\left(y^{(i)} = c\right) \tag{1}$$

where $\mathbb{I}(\cdot)$ is the indicator function. The predicted label $\hat{y}(\mathbf{x})$ corresponds to the local majority vote:

$$\hat{y}(\mathbf{x}) = \arg \max_{c \in \{1,\dots,C\}} P(Y = c \mid \mathbf{X} = \mathbf{x}) = \arg \max_{c \in \{1,\dots,C\}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbb{I}\left(y^{(i)} = c\right) \tag{2}$$

For **Regression**, k-NN estimates the target value $y(\mathbf{x}) \in \mathbb{R}$ as the expected value over the local neighborhood:

$$\hat{y}(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}] \approx \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y^{(i)} \tag{3}$$

2 Metric Spaces and Distance Geometry

Because k-NN relies entirely on neighborhood proximity, the choice of distance metric defines the geometry of the hypothesis space.

2.1 Formal Metric Axioms

A distance function $d : \mathcal{X} \times \mathcal{X} \to \mathbb{R}_{\geq 0}$ converts feature space $\mathcal{X}$ into a formal **Metric Space** $(\mathcal{X}, d)$ if and only if it satisfies four fundamental axioms for all $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathcal{X}$:

1. **Non-Negativity:** $d(\mathbf{x}, \mathbf{y}) \geq 0$.
2. **Identity of Indiscernibles:** $d(\mathbf{x}, \mathbf{y}) = 0 \iff \mathbf{x} = \mathbf{y}$.
3. **Symmetry:** $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$.
4. **Triangle Inequality:** $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$.

2.2 Minkowski Metric Family (L_p Norms)

The most common metric family for continuous vector spaces $\mathbb{R}^d$ is the **Minkowski Distance (L_p Norm)**:

$$D_p(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_p = \left(\sum_{j=1}^{d} |x_j - z_j|^p \right)^{1/p}, \quad p \geq 1 \tag{4}$$

1. Manhattan Distance (L_1 Norm, $p = 1$)

$$D_1(\mathbf{x}, \mathbf{z}) = \sum_{j=1}^{d} |x_j - z_j| \tag{5}$$

Measures distance along grid axes. Suitable for high-dimensional or sparse feature spaces where independent feature differences accumulate linearly without quadratic inflation.

2. Euclidean Distance (L_2 Norm, $p = 2$)

$$D_2(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{d} (x_j - z_j)^2} = \sqrt{(\mathbf{x} - \mathbf{z})^T (\mathbf{x} - \mathbf{z})} \tag{6}$$

The standard isotropic physical distance. Sensitive to feature scale variations, requiring feature standardization (z-score normalization) prior to evaluation.

3. Chebyshev Distance (L_∞ Norm, $p \to \infty$)

$$D_\infty(\mathbf{x}, \mathbf{z}) = \max_{j \in \{1, \ldots, d\}} |x_j - z_j| \tag{7}$$

Measures the maximum single feature discrepancy across all d dimensions.

2.3 Mahalanobis Distance for Correlated Feature Spaces

When features exhibit inter-dimensional correlations or unequal variances, the isotropic L_2 norm distorts proximity. The **Mahalanobis Distance** incorporates the covariance structure $\mathbf{\Sigma}$:

$$D_M(\mathbf{x}, \mathbf{z}) = \sqrt{(\mathbf{x} - \mathbf{z})^T \mathbf{\Sigma}^{-1} (\mathbf{x} - \mathbf{z})} \tag{8}$$

This scales distances along principal axes of variance, converting elliptical neighborhood contours into spherical metrics.

3 Voronoi Tessellations and Decision Geometries

For $k = 1$, the decision boundary constructed by nearest-neighbor classification forms a convex cell structure known as a **Voronoi Tessellation (Dirichlet Decomposition)**.

Definition 3.1 (**Voronoi Cell**). Given a training set $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(N)}, y^{(N)})\}$, the **Voronoi Cell** $V_i \subset \mathbb{R}^d$ associated with training sample $\mathbf{x}^{(i)}$ is the closed subregion of space closer to $\mathbf{x}^{(i)}$ than to any other sample in $\mathcal{D}$:

$$V_i = \left\{ \mathbf{x} \in \mathbb{R}^d \;\middle|\; d(\mathbf{x}, \mathbf{x}^{(i)}) \leq d(\mathbf{x}, \mathbf{x}^{(j)}) \quad \forall j \neq i \right\} \tag{9}$$

3.1 Hyperplane Boundaries of 1-NN

For any pair of distinct training points $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(j)}$, the locus of points equidistant from both under L_2 metric is a perpendicular bisecting hyperplane H_{ij}:

$$H_{ij} = \left\{ \mathbf{x} \in \mathbb{R}^d \;\middle|\; \|\mathbf{x} - \mathbf{x}^{(i)}\|_2^2 = \|\mathbf{x} - \mathbf{x}^{(j)}\|_2^2 \right\} \tag{10}$$

Expanding the quadratic terms:

$$\mathbf{x}^T\mathbf{x} - 2(\mathbf{x}^{(i)})^T\mathbf{x} + \|\mathbf{x}^{(i)}\|_2^2 = \mathbf{x}^T\mathbf{x} - 2(\mathbf{x}^{(j)})^T\mathbf{x} + \|\mathbf{x}^{(j)}\|_2^2 \tag{11}$$

$$2(\mathbf{x}^{(j)} - \mathbf{x}^{(i)})^T\mathbf{x} + \left(\|\mathbf{x}^{(i)}\|_2^2 - \|\mathbf{x}^{(j)}\|_2^2 \right) = 0 \tag{12}$$

Each Voronoi cell V_i is formed by the intersection of $N - 1$ linear half-spaces:

$$V_i = \bigcap_{j \neq i} \left\{ \mathbf{x} \in \mathbb{R}^d \;\middle|\; 2(\mathbf{x}^{(j)} - \mathbf{x}^{(i)})^T\mathbf{x} + \|\mathbf{x}^{(i)}\|_2^2 - \|\mathbf{x}^{(j)}\|_2^2 \leq 0 \right\} \tag{13}$$

Because the intersection of closed half-spaces yields a **convex polyhedron**, the decision boundary separating different classes in 1-NN consists of a non-linear, piecewise linear polygonal surface.

1-NN Voronoi Tessellation and Piecewise Decision Boundary

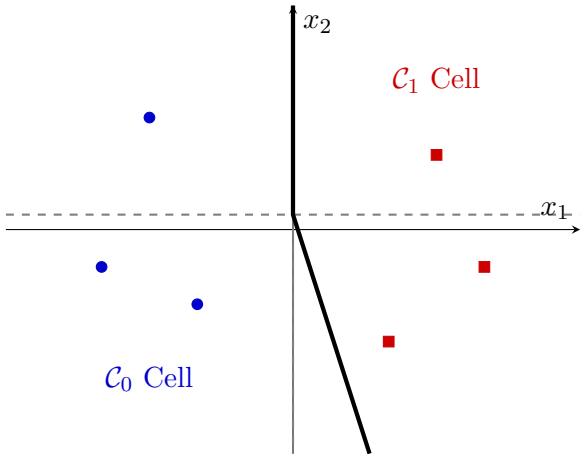

Figure 1: The 1-NN algorithm partitions space into convex Voronoi cells. Boundaries between cells of differing classes assemble into a piecewise linear surface.

4 Asymptotic Error Analysis and the Cover-Hart Theorem

A foundational theoretical result in statistical learning theory, proved by Thomas Cover and Peter Hart in 1967, establishes that the error rate of the simple 1-nearest neighbor classifier is upper-bounded by twice the optimal **Bayes Error Rate** as $N \to \infty$.

4.1 The Bayes Optimal Classifier

Let R^* denote the minimum possible generalization error achievable by any decision rule (the Bayes Risk):

$$R^* = \mathbb{E}_{\mathbf{X}}\left[1 - \max_{c \in \{1,\dots,C\}} P(Y = c \mid \mathbf{X})\right] \tag{14}$$

For a binary classification problem ($C = 2$) with $p(\mathbf{x}) = P(Y = 1 \mid \mathbf{X} = \mathbf{x})$, the local Bayes Risk simplifies to:

$$r^*(\mathbf{x}) = \min\left(p(\mathbf{x}),\, 1 - p(\mathbf{x})\right) = p(\mathbf{x})(1 - p(\mathbf{x})) \quad (\text{tight bound: } r^* \geq p(1-p)) \tag{15}$$

Overall Bayes Risk is:

$$R^* = \int \min\left(p(\mathbf{x}), 1 - p(\mathbf{x})\right) p(\mathbf{x}) d\mathbf{x} \tag{16}$$

4.2 Statement and Proof of the Cover-Hart Theorem

Theorem 4.1 (Cover-Hart Theorem, 1967). Let $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ be an i.i.d. dataset drawn from a continuous probability distribution where $P(Y \mid \mathbf{X})$ is smooth (Lipschitz continuous). Let $R_{\text{1-NN}}^{(N)}$ be the expected risk of the 1-NN classifier over N samples. Then, as $N \to \infty$:

$$R^* \leq R_{\text{1-NN}}^{(\infty)} \leq 2R^*\left(1 - \frac{C}{C-1}R^*\right) \leq 2R^* \tag{17}$$

where C is the number of classes.

Proof Sketch for $C = 2$ Classes. Let $\mathbf{x}$ be a query point and $\mathbf{x}^{(N)'} \in \mathcal{D}$ be its nearest neighbor among N training samples. As $N \to \infty$, the distance $\|\mathbf{x} - \mathbf{x}^{(N)'}\| \to 0$ almost surely, which implies:

$$p(\mathbf{x}^{(N)'}) = P(Y = 1 \mid \mathbf{X} = \mathbf{x}^{(N)'}) \to p(\mathbf{x}) = P(Y = 1 \mid \mathbf{X} = \mathbf{x}) \tag{18}$$

The 1-NN algorithm assigns label $\hat{y} = y^{(N)'}$. An error occurs at $\mathbf{x}$ if either:

1. True label $Y = 1$ (prob. $p(\mathbf{x})$) and Neighbor label $Y^{(N)'} = 0$ (prob. $1 - p(\mathbf{x}^{(N)'})$).
2. True label $Y = 0$ (prob. $1 - p(\mathbf{x})$) and Neighbor label $Y^{(N)'} = 1$ (prob. $p(\mathbf{x}^{(N)'})$).

Because Y and $Y^{(N)'}$ are conditionally independent given their feature vectors:

$$\begin{aligned} r_{\text{1-NN}}(\mathbf{x}) &= \lim_{N\to\infty}\left[p(\mathbf{x})\left(1 - p(\mathbf{x}^{(N)'})\right) + (1 - p(\mathbf{x}))p(\mathbf{x}^{(N)'})\right] \\ &= p(\mathbf{x})(1 - p(\mathbf{x})) + (1 - p(\mathbf{x}))p(\mathbf{x}) \\ &= 2p(\mathbf{x})(1 - p(\mathbf{x})) \end{aligned} \tag{19}$$

Recall that local Bayes risk is $r^*(\mathbf{x}) = \min(p(\mathbf{x}), 1 - p(\mathbf{x}))$. Expressing $p(\mathbf{x})(1 - p(\mathbf{x}))$ in terms of $r^*(\mathbf{x})$:

$$p(\mathbf{x})(1 - p(\mathbf{x})) = r^*(\mathbf{x})(1 - r^*(\mathbf{x})) \tag{20}$$

Substituting this back into the expected 1-NN risk:

$$r_{\text{1-NN}}(\mathbf{x}) = 2r^*(\mathbf{x})(1 - r^*(\mathbf{x})) \tag{21}$$

Integrating over the full feature space $\mathcal{X}$:

$$R_{\text{1-NN}}^{(\infty)} = \int 2r^*(\mathbf{x})(1 - r^*(\mathbf{x}))p(\mathbf{x})d\mathbf{x} = 2R^* - 2\int (r^*(\mathbf{x}))^2 p(\mathbf{x})d\mathbf{x} \tag{22}$$

Applying Jensen's Inequality ($\mathbb{E}[(r^*)^2] \geq (\mathbb{E}[r^*])^2 = (R^*)^2$):

$$R_{\text{1-NN}}^{(\infty)} \leq 2R^* - 2(R^*)^2 = 2R^*(1 - R^*) \leq 2R^* \tag{23}$$

This completes the proof. □

Theorem 4.2 (**Consistency of k-NN**). If $k \to \infty$ and $\frac{k}{N} \to 0$ as $N \to \infty$, the expected risk of the k-NN classifier converges directly to the Bayes Optimal Risk:

$$\lim_{\substack{N\to\infty, k\to\infty \\ k/N\to 0}} R_{k\text{-NN}}^{(N)} = R^* \tag{24}$$

This property makes k-NN a **Universally Consistent** classifier.

5 The Curse of Dimensionality

While theoretical asymptotic properties are strong, performance degrades in high-dimensional feature spaces ($\mathbb{R}^d$ for $d > 20$) due to the **Curse of Dimensionality** (Bellman, 1961).

5.1 Geometric Expansion of Hypervolume

Consider a unit hypercube $[-1, 1]^d$ in d dimensions with volume $V_{\text{cube}} = 2^d$. An inscribed hypersphere of radius $r = 1$ has volume:

$$V_{\text{sphere}}(d, 1) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2} + 1\right)} \tag{25}$$

Evaluating the volume ratio of the inscribed sphere to the bounding hypercube:

$$\lim_{d\to\infty} \frac{V_{\text{sphere}}(d, 1)}{V_{\text{cube}}(d, 1)} = \lim_{d\to\infty} \frac{\pi^{d/2}}{2^d \Gamma\left(\frac{d}{2} + 1\right)} = 0 \tag{26}$$

In high dimensions, almost all volume of a hypercube concentrates in its outer corners, leaving the interior sphere empty.

5.2 Empty Space Phenomenon and Neighborhood Dilation

To capture a fraction $q = \frac{k}{N}$ of data points uniformly distributed in a unit hypercube $[0, 1]^d$, the local neighborhood must expand its sub-cube edge length $e_d(q)$:

$$V_{\text{neighborhood}} = (e_d(q))^d = q \implies e_d(q) = q^{1/d} \tag{27}$$

For $d = 10$, to capture just 1% of the data ($q = 0.01$):

$$e_{10}(0.01) = (0.01)^{1/10} \approx 0.631 \tag{28}$$

To capture 1% of the dataset, the "local" neighborhood must cover 63.1% of the entire feature range along every dimension. The locality assumption ($d(\mathbf{x}, \mathbf{x}') \to 0$) breaks down.

5.3 Distance Concentration Phenomenon

As dimensionality d increases, the distance between any pair of points converges to a constant value. Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ be i.i.d. random vectors:

Theorem 5.1 (**Distance Concentration (Beyer et al., 1999)**). Under mild statistical conditions, the relative difference between the maximum and minimum distance from a query point $\mathbf{x}$ to all other points vanishes as $d \to \infty$:

$$\lim_{d\to\infty} \frac{D_{\max} - D_{\min}}{D_{\min}} = 0 \tag{29}$$

In high dimensions, concept of "nearest neighbor" becomes meaningless because all points become nearly equidistant from the query point.

6 Physical Analogies: Wigner-Seitz Cells and Lattice Gas Dynamics

k-NN mathematics connects directly to solid-state physics, crystallography, and computational fluid mechanics.

6.1 Wigner-Seitz Primitive Cells in Solid-State Physics

In condensed matter physics, a crystal lattice consists of periodic arrangements of atoms in space.

Definition 6.1 (**Wigner-Seitz Primitive Cell**). The **Wigner-Seitz Cell** surrounding a lattice node $\mathbf{R}_i$ is defined as the geometric region of space closer to $\mathbf{R}_i$ than to any neighboring lattice node $\mathbf{R}_j$:

$$\Omega_{\text{WS}} = \left\{ \mathbf{r} \in \mathbb{R}^3 \;\middle|\; \|\mathbf{r} - \mathbf{R}_i\|_2 \leq \|\mathbf{r} - \mathbf{R}_j\|_2 \quad \forall j \neq i \right\} \tag{30}$$

The Wigner-Seitz cell of a crystal lattice is mathematically identical to a **1-NN Voronoi Cell**. In solid-state theory, electron wavefunctions $\psi(\mathbf{r})$ are computed within these Dirichlet domains, matching local density estimation in instance-based machine learning.

6.2 Short-Range Interaction Cutoffs in Molecular Dynamics

In molecular dynamics simulations (such as Lennard-Jones or Coulomb fluids), calculating pairwise interactions across N particles scales as $O(N^2)$. To reduce computational overhead, interactions are truncated using a short-range **Nearest-Neighbor Cutoff Potential**:

$$V_{\text{total}}(\mathbf{r}_i) = \sum_{j \in \mathcal{N}_k(\mathbf{r}_i)} V\left(\|\mathbf{r}_i - \mathbf{r}_j\|\right) \tag{31}$$

k-NN operates as a truncated potential model where field strength or class probability depends only on short-range interactions within a localized force radius.

7 Computational Acceleration: KD-Trees and Ball-Trees

Evaluating N distances for every query becomes slow for large N. Spatial indexing structures accelerate query times.

7.1 KD-Trees (k-Dimensional Trees)

A **KD-Tree** is a binary space-partitioning tree that recursively bisects feature space along axis-aligned hyperplanes:

1. Select feature dimension j with maximum variance. 2. Find the median value along dimension j among current points. 3. Split points into left sub-tree ($x_j <$ median) and right sub-tree ($x_j \geq$ median). 4. Repeat recursively until leaf nodes contain fewer than M_{leaf} points.

Search query complexity drops from $O(Nd)$ to **$O(d \log N)$** on average. However, when $d > 20$, backtracking requirements cause KD-Trees to degrade back to brute-force $O(Nd)$ search.

7.2 Ball-Trees

For high-dimensional spaces, **Ball-Trees** partition space into nested hyperspheres (balls) defined by center $\mathbf{c}$ and radius r:

$$B = \{\mathbf{x} \in \mathbb{R}^d \mid \|\mathbf{x} - \mathbf{c}\|_2 \leq r\} \tag{32}$$

Using the triangle inequality, Ball-Trees prune search branches efficiently even when feature dimensions are correlated or non-Euclidean.

8 Production-Grade Vectorized NumPy Implementation

The following Python class implements a unified k-NN engine supporting both classification and regression, distance weighting, and $O(N)$ selection via 'np.argpartition'.

```python
import numpy as np

class UniversalKNNEngine:
    """
    Production-grade k-Nearest Neighbors (k-NN) Engine supporting Classification,
    Regression, Minkowski p-norms, Distance Weighting, and Vectorized Distance Calculations.
    """
    def __init__(self, n_neighbors: int = 5, metric: str = 'minkowski',
                 p: float = 2.0, weights: str = 'uniform', mode: str = 'classification'):
        self.n_neighbors = n_neighbors
        self.metric = metric
        self.p = p
        self.weights = weights
        self.mode = mode

        self.X_train = None
        self.y_train = None
        self.classes_ = None

    def fit(self, X: np.ndarray, y: np.ndarray):
        """Stores training instances (O(1) Lazy Learning step)."""
        self.X_train = np.asarray(X, dtype=np.float64)
        self.y_train = np.asarray(y)

        if self.mode == 'classification':
            self.classes_ = np.unique(self.y_train)

        return self

    def _compute_pairwise_distances(self, X_query: np.ndarray) -> np.ndarray:
        """
        Computes pairwise distance matrix D (N_query x N_train).
        Uses optimized quadratic expansion for L2 norm: ||a - b||^2 = ||a||^2 + ||b||^2 - 2<a, b>
        """
        N_query = X_query.shape[0]
        N_train = self.X_train.shape[0]

        if self.metric == 'minkowski' and self.p == 2.0:
            # Fast Vectorized Euclidean Distance
            query_sq = np.sum(X_query**2, axis=1, keepdims=True)      # (N_query, 1)
            train_sq = np.sum(self.X_train**2, axis=1, keepdims=True).T # (1, N_train)
            dot_product = X_query @ self.X_train.T                    # (N_query, N_train)

            dist_sq = query_sq + train_sq - 2.0 * dot_product
            dist_sq = np.maximum(dist_sq, 0.0) # Prevent negative values due to float precision
            return np.sqrt(dist_sq)
```

```
47
48          elif self.metric == 'minkowski' and self.p == 1.0:
49              # Vectorized Manhattan Distance L1
50              # Broadcasting over (N_query, 1, d) - (1, N_train, d)
51              return np.sum(np.abs(X_query[:, np.newaxis, :] - self.X_train[
    np.newaxis, :, :]), axis=2)
52
53          else:
54              # General Minkowski L_p Metric
55              diff = np.abs(X_query[:, np.newaxis, :] - self.X_train[np.
    newaxis, :, :])
56              return np.sum(diff**self.p, axis=2)**(1.0 / self.p)
57
58      def predict_proba(self, X_query: np.ndarray) -> np.ndarray:
59          """Computes posterior class probabilities for classification."""
60          if self.mode != 'classification':
61              raise ValueError("predict_proba is available only for mode='
    classification'.")
62
63          X_q = np.asarray(X_query, dtype=np.float64)
64          N_query = X_q.shape[0]
65          K = self.n_neighbors
66          num_classes = len(self.classes_)
67
68          # 1. Compute Pairwise Distance Matrix (N_query x N_train)
69          D = self._compute_pairwise_distances(X_q)
70
71          # 2. Extract Top-K Nearest Neighbor Indices using O(N) Partial Sort
72          top_k_indices = np.argpartition(D, K - 1, axis=1)[:, :K]
73
74          probabilities = np.zeros((N_query, num_classes))
75
76          for i in range(N_query):
77              k_idx = top_k_indices[i]
78              k_dists = D[i, k_idx]
79              k_labels = self.y_train[k_idx]
80
81              # Compute Weights: 'uniform' or 'distance' inverse weighting
82              if self.weights == 'distance':
83                  # Avoid division by zero for exact match points
84                  weights_i = 1.0 / (k_dists + 1e-10)
85              else:
86                  weights_i = np.ones(K)
87
88              # Normalize weights
89              sum_weights = np.sum(weights_i)
90
91              # Accumulate class probabilities
92              for c_idx, c_val in enumerate(self.classes_):
93                  matching_mask = (k_labels == c_val)
94                  probabilities[i, c_idx] = np.sum(weights_i[matching_mask])
    / sum_weights
95
96          return probabilities
97
98      def predict(self, X_query: np.ndarray) -> np.ndarray:
99          """Predicts discrete labels (classification) or continuous values (
    regression)."""
100         X_q = np.asarray(X_query, dtype=np.float64)
101
```

```
        if self.mode == 'classification':
            probs = self.predict_proba(X_q)
            max_indices = np.argmax(probs, axis=1)
            return self.classes_[max_indices]

        elif self.mode == 'regression':
            N_query = X_q.shape[0]
            K = self.n_neighbors

            D = self._compute_pairwise_distances(X_q)
            top_k_indices = np.argpartition(D, K - 1, axis=1)[:, :K]

            predictions = np.zeros(N_query)

            for i in range(N_query):
                k_idx = top_k_indices[i]
                k_dists = D[i, k_idx]
                k_values = self.y_train[k_idx]

                if self.weights == 'distance':
                    weights_i = 1.0 / (k_dists + 1e-10)
                    predictions[i] = np.sum(weights_i * k_values) / np.sum(weights_i)
                else:
                    predictions[i] = np.mean(k_values)

            return predictions


if __name__ == "__main__":
    np.random.seed(42)

    # 1. Validation Run: Classification Mode
    N_train, N_test, d_feat = 500, 100, 5
    X_tr = np.random.randn(N_train, d_feat)
    y_tr = (X_tr[:, 0] + X_tr[:, 1] * 0.5 > 0).astype(int)

    X_te = np.random.randn(N_test, d_feat)
    y_te = (X_te[:, 0] + X_te[:, 1] * 0.5 > 0).astype(int)

    knn_clf = UniversalKNNEngine(n_neighbors=7, metric='minkowski', p=2.0, weights='distance', mode='classification')
    knn_clf.fit(X_tr, y_tr)

    preds_clf = knn_clf.predict(X_te)
    acc = np.mean(preds_clf == y_te)

    print(f"=== k-NN CLASSIFICATION ACCURACY: {acc * 100:.2f}% ===")
    print(f"Evaluated Test Samples: {N_test}")

    # 2. Validation Run: Regression Mode
    y_tr_reg = np.sin(X_tr[:, 0]) + 0.1 * np.random.randn(N_train)
    y_te_reg = np.sin(X_te[:, 0]) + 0.1 * np.random.randn(N_test)

    knn_reg = UniversalKNNEngine(n_neighbors=5, metric='minkowski', p=2.0, weights='uniform', mode='regression')
    knn_reg.fit(X_tr, y_tr_reg)

    preds_reg = knn_reg.predict(X_te)
    mse = np.mean((preds_reg - y_te_reg)**2)
```

```
159
160     print(f"\n=== k-NN REGRESSION MEAN SQUARED ERROR: {mse:.4f} ===")
```

Listing 1: Vectorized NumPy implementation of k-Nearest Neighbors Classifier and Regressor.